8th International Conference on Engineering Research, Innovation and Education
School of Applied Sciences Technology, SUST, Sylhet

# Survey of Machine Learning Techniques for Detecting Buffer Overflow Vulnerabilities

Emran Kanaan, Syeda Sadia Alam, *Mst Shapna Akter

Department of Computer Science and Engineering,
Oakland University,
318 Meadow Brook Road, Rochester Hills, MI 48309-4454, USA
Emails: {emrankanaan, akter}@oakland.edu
Computer Sceince Departement. Metropolitan University,
Pirer Bazar,Sylhet,Bangladesh
Email:syeda.alam.juti@gmail.com

***Mst Shapna Akter**

**Keywords:**
Buffer Overflow,
Software Security,
Machine Learning,
Deep Learning,
Static Analysis,
Dynamic Analysis,
Vulnerability.

**Abstract:** Buffer overflow (CWE-120) vulnerabilities pose serious financial and operational risks due to their frequent exploitation by attackers. Recent research has applied both machine learning (ML) and deep learning (DL) methods to detect such flaws in real-world scenarios. This survey reviews detection strategies from 2013 to 2024, tracing the shift from traditional ML techniques to advanced DL architectures that integrate multiple data sources, including source code, binary files, and system logs. Despite significant progress, challenges remain related to limited data availability, interpretability, and adaptability across diverse environments. To address these issues, future work should prioritize the development of standardized datasets, semi-supervised learning frameworks, and ethical considerations, ultimately strengthening AI-driven cybersecurity solutions.

## 1 Introduction

Nowadays, most of our daily activities depend on technology and software. Software is integrated into a wide range of systems, from small household devices such as microwaves and ovens to larger, complex systems like cars and planes. As more features are added to both software and hardware, such as processors, caches, and memory, these systems become more capable. Consequently, software grows in both size and complexity. The rise in software complexity has led to an increase in the number of code vulnerabilities reported over the past years Software vulnerabilities are responsible for many real-world disastrous attacks (Hanif and Maffeis [2022]). One of the primary causes of software vulnerabilities is flaws or weaknesses within program code, particularly buffer overflow vulnerabilities (CWE-120), which allow attackers to manipulate memory space to execute malicious actions (Byun et al. [2020]). Such weaknesses open the door for hackers to exploit the system. Almost any programming language can contain hidden vulnerabilities if developed without adequate security testing. Some languages, like SafeR, Rust, and Go, have built-in features for input validation and memory safety, which help reduce the risk of certain types of weaknesses and place them in the category of safer programming languages (Li et al. [2023]). On the other hand, languages like C and C++ are considered less safe . These low-level languages lack built-in protections for memory safety, thread safety, and type management, making buffer overflow vulnerabilities more accessible. Code bugs and missing safety checks are common in C and C++ programs (Alenezi and Tsokos [2020]). Despite these security concerns, C and C++ remain widely used due to the large base of existing

code and their low-level features, which are essential for high-performance systems and specific memory use (Ji et al. [2018]). Because of these benefits, C and C++ consistently rank among the top five most popular programming languages, according to the TIOBE index.To address the issue of software vulnerabilities, developers and researchers have collaborated to better understand and categorize software weaknesses. One significant outcome of these efforts is the Common Weakness Enumeration (CWE), a list that highlights the top hardware and software weaknesses (Hanif and Maffeis [2022]). Updated annually, this list provides guidance on identifying and addressing critical vulnerabilities, including buffer overflow vulnerabilities, which remain among the most exploited flaws in software security. When source code is accessible, traditional manual code reviews are often conducted to identify defects. Although still commonly used, this approach can be time-consuming and labor-intensive. To improve efficiency, researchers have developed various automated tools and techniques to detect vulnerabilities in high-level source code (Li et al. [2019]). Among the earliest tools were PC-Lint and Lint, which were designed to detect errors and defects in C and C++ programs (Huang et al. [2019]). Today, more advanced Static Application Security Testing (SAST) tools such as Klocwork, Coverity, Flawfinder, CodeSonar, and SonarQube are widely used. These tools are often employed in the later stages of development to enhance software quality, especially for identifying vulnerabilities in source code.

Software often distributed in binary form lacks original source code, complicating vulnerability detection. Analyzing binary code poses greater challenges than source code analysis, and SAST tools, limited to the binary's scope, cannot ensure complete security (Li et al. [2019]). The effectiveness of these tools is further reduced by the lack of high-quality, standardized datasets essential for training robust detection systems. Future research must enhance detection capabilities to adapt to rapid software and threat landscape changes, ensuring the mitigation of emerging buffer overflow vulnerabilitiese. After this introduction, Section 2 explains the methodology used for the literature review, Section 3 provides a thematic analysis of the collected literature, Section 4 discusses the challenges found and suggests directions for future research. Finally, Section 5 a summary of the findings and their implications for future work in the field.

# 2 Literature Review

Machine learning (ML) and deep learning (DL) have helped improve the detection of software vulnerabilities like buffer overflow (CWE-120). This section reviews important studies, covering their methods, model types, datasets, accuracy, and challenges they found.

## 2.1 Traditional Approaches

vulnerability detection in software systems has relied on two main approaches: static and dynamic analysis. Static analysis tools, such as Lint, inspect source code without execution, enabling early identification of programming errors that may lead to security breaches. In contrast, dynamic analysis evaluates software at runtime under controlled conditions, often revealing vulnerabilities overlooked by static methods. This runtime perspective provides a practical assessment of the software's behavior and security posture. Notably, the integration of machine learning techniques particularly Support Vector Machines (SVMs) into dynamic analysis has significantly enhanced detection capabilities. For instance, applying SVMs to the National Vulnerability Database (NVD) has yielded approximately 78% accuracy, illustrating the potential of machine learning to improve traditional analysis methods (Huang et al. [2019]).

Table 1: An example of how we filter the papers based on our criteria.

| Paper ID | Topic Relevance | Method Quality | Application Scope | ML/DL Focus | Empirical Evidence | Included/Excluded |
|---|---|---|---|---|---|---|
| (Li [2018]) | ✓ | ✓ | ✓ | ✓ | ✓ | Included |
| (Byun et al. [2020]) | ✗ | ✗ | ✗ | ✗ | ✗ | Excluded |
| Li et al. [2019]) | ✗ | ✓ | ✗ | ✓ | ✗ | Included |
| (Kumar and Mallipeddi [2022]) | ✓ | ✓ | ✓ | ✓ | ✓ | Included |
| (Li et al. [2023]) | ✓ | ✓ | ✗ | ✗ | ✗ | Excluded |
| (Li et al. [2023]) | ✗ | ✗ | ✗ | ✗ | ✗ | Excluded |

## 2.2 Machine Learning Approaches

In recent years, machine learning (ML) has become popular in detecting buffer overflow vulnerabilities. Traditional models like Decision Trees and Random Forests achieved up to 80% accuracy on datasets like MITRE CWE (Hanif and Maffeis [2022]). Advances in neural networks brought about significant improvements; models such as CNNs, RNNs with LSTMs, and GNNs have achieved accuracies ranging from 84% to 89% across various datasets including DARPA CGC, Juliet Test Suite, and OSV (Das et al. [2021]). Hybrid models and unsupervised techniques like autoencoders further demonstrated high efficacy, notably 88% accuracy with CNN-LSTM architectures on VulDeePecker and 82% in unsupervised settings on SARD (Alenezi and Tsokos [2020]). Recent innovations, such as neural embeddings for source code analysis, have shown potential by enhancing detection of CWE vulnerabilities, with novel data representation techniques like BoW and Word2Vec further improving accuracy rates on diverse datasets (Byun et al. [2020]). A comprehensive survey underscores the complexity of cyber threats and the critical need for sophisticated ML defenses (Ji et al. [2018]). Overall, the shift towards high-accuracy, proactive detection mechanisms highlights the ongoing evolution and necessity of innovation in cybersecurity

# 3 Finding and challenges.

This section explains the step-by-step approach we used to conduct a focused survey of machine learning (ML) and deep learning (DL) applications in buffer overflow vulnerability detection. Our method included three main steps: finding relevant literature, setting criteria for including or excluding studies, and summarizing and analyzing the data. These steps were carefully planned to ensure a thorough and unbiased review of existing research in this specific area.

## 3.1 Literature Review Process

To carry out a comprehensive literature review, we searched several major academic databases, including IEEE Xplore, ACM Digital Library, ScienceDirect, and SpringerLink. These databases were selected due they are known for high-quality, peer-reviewed content, which is important for maintaining academic standards (Li [2018]). We used a variety of keywords and phrases specifically related to buffer overflow vulnerabilities, such as machine learning, deep learning, buffer overflow detection, cybersecurity, code analysis, memory safety, and automated detection tools. This approach helped us capture a broad range of studies while keeping the focus on recent advancements relevant to buffer overflow detection(Kumar and Mallipeddi [2022]).

## 3.2 Inclusion and Exclusion Criteria

To ensure high-quality and relevant research in our systematic review on ML and DL applications for buffer overflow detection, we set precise inclusion and exclusion criteria. Studies were included if they explicitly applied ML or DL to buffer overflow detection, supported

by practical implementations and empirical data, demonstrated a methodology with clear objectives, documented procedures, and reproducible findings, and addressed significant trends, challenges, or real-world applications related to ML/DL for buffer overflow vulnerabilities (Ji et al. [2018]). Studies were excluded if they did not focus primarily on buffer overflow detection using ML/DL, offered only theoretical concepts without empirical validation, or concentrated on unrelated areas like hardware vulnerabilities without clear ML/DL relevance to software security (Kumar and Mallipeddi [2022]). These criteria reflect common guidelines for systematic reviews in cybersecurity (Huang et al. [2019]), ensuring our survey remains focused on pertinent, methodologically sound research. Additionally, to further enrich our review of current trends, we included studies exploring the potential of Large Language Models (LLMs) in buffer overflow scenarios, recognizing their capability to enhance detection accuracy and reliability in complex software systems

Table 2: Summary of Review Categories, Findings, and Challenges

| Category | Technique | Source | Findings | Challenges |
|---|---|---|---|---|
| Scope of Review | ML/DL for Buffer Overflow | [19], [20] | Covers 2013 to 2024, focusing specifically on ML/DL advancements for buffer overflow detection. | Prior surveys cover up to 2022 with a general focus on software vulnerabilities. |
| Technologies Covered | SVM, Decision Trees, Transformers, GNNs | [21], [22], [23], [24] | Includes both traditional and advanced DL models tailored for buffer overflow tasks. | Existing literature primarily discusses traditional ML and basic DL for broader cybersecurity issues. |
| Data Types Analyzed | Source Code, Binary Files, Memory Dumps, Stack Traces, System Logs | [25], [26], [27], [28] | Utilizes diverse data types relevant to buffer overflow, enhancing detection capabilities. | Most studies focus on source code and binary files, neglecting other data types. |
| Integration of Findings | Data Source Integration | [27], [28] | Integrates findings from various data sources to assess ML/DL effectiveness for buffer overflow. | Previous studies often treat data sources separately, limiting comprehensive analysis. |
| Practical Applications | Case Studies | [29], [30] | Emphasizes real-world applications, including industry implementations for buffer overflow detection. | Discussions in prior studies are mostly theoretical with limited real-world application focus. |
| Future Research Directions | Specialized Datasets, Unsupervised Learning, Interpretability | [21], [22], [23], [24], [25], [27], [28] | Proposes new datasets and methods to enhance interpretability and effectiveness of ML/DL models. | Existing guidance is generally broad, lacking specific actionable recommendations for buffer overflow. |

## 3.3 Data Synthesis and Analysis

We methodically extracted key information from studies meeting our criteria, including authors, publication years, objectives, methods, and findings relevant to buffer overflow vulnerabilities (Huang et al. [2019]). Using thematic analysis, we organized the data, categorizing studies by the ML/DL techniques they employed for detecting buffer overflows (Hanif et al. [2021]). We also performed citation analysis and monitored publication trends to assess the evolution of these technologies and identify research gaps. This comprehensive analysis not only highlighted current trends but also areas needing further exploration.

Our survey, covering ML/DL applications for buffer overflow detection from 2013 to 2024, addresses a niche overlooked by broader vulnerability studies. We analyzed various data types, such as memory dumps and stack traces, offering practical insights and real-world applications for deploying ML/DL solutions. Additionally, we suggested creating specific datasets for buffer overflow to propel the field forward.

# 4 Outcomes of the Survey

As software becomes integral to daily life, the rise in security vulnerabilities has driven increased research into machine learning (ML) and deep learning (DL) for vulnerability detection. This section summarizes key findings from 2013 to 2024, highlighting the evolution of ML/DL technologies, their strengths, and areas needing improvement.

## 4.1 Overview of Methods and Models

Our survey reviews various ML and DL methods for detecting software vulnerabilities. Initially focused on simpler models like Decision Trees and Support Vector Machines (SVMs), recent research has shifted towards advanced DL models such as Convolutional Neural Networks (CNNs) and Transformers, which are better equipped to handle complex data (Li et al. [2023], This transition underscores the increasing capability of models to address more sophisticated software threats.

## 4.2 Gaps in Current Research

Despite advancements, significant research gaps hinder the effectiveness of machine learning and deep learning models in practical applications. Standardization issues arise as the lack of uniform datasets and metrics complicates model comparison(Kumar and Mallipeddi [2022]). Furthermore (Filus and Domańska [2023]) point out that high-quality, annotated datasets remain scarce, impacting model training adversely. Additionally, model generalizability is a critical concern; many models perform poorly outside their training environments, which limits their broader applicability

Table 3: Summary of Research Gaps and Challenges

| Gap | Description | Challenge | Reference |
|---|---|---|---|
| Standardization | Absence of common datasets for benchmarking | Complicates comparability | (Ji et al. [2018]), (Ji et al. [2018]) |
| Data Scarcity & Quality | Limited availability of quality, annotated datasets | Impacts training accuracy | (Ji et al. [2018]), (Li et al. [2019]) |
| Model Generalizability | Models' limited adaptability | Restricts deployment | (Bilgin et al. [2020]), (Butt et al. [2022]) |
| Interpretability | Complexity in model predictions | Reduces usability | Hanif et al. [2021], Li [2018] |
| Integration with Dev Tools | Difficulty integrating ML models into development processes | Hinders adoption | (Filus and Domańska [2023]), (Alenezi and Tsokos [2020]) |

## 4.3 Suggested Future Directions

To bridge these gaps, future research should focus on:

- **Standardized Datasets:** Develop shared, high-quality datasets to facilitate model comparison and improvement (Kumar and Mallipeddi [2022]).
- **Improving Model Robustness:** Enhance models' adaptability across various software environments using methods like adversarial and transfer learning (Hanif et al. [2021]).

This survey underscores the progress in ML/DL for vulnerability detection and the challenges still to be addressed. Enhancing standardization, data quality, and model adaptability will advance the efficacy of ML/DL in cybersecurity.

# 5 Conclusion

This survey reviewed machine learning (ML) and deep learning (DL) approaches for detecting buffer overflow vulnerabilities from 2013 to 2024. We observed a shift from basic ML techniques to more advanced DL models like CNNs and Transformers, which handle complex data more effectively. Integrating various data sources such as source code, binaries, and logs has improved detection accuracy, while the move toward real-time detection reflects a growing

need for faster responses to threats. However, there are still challenges. The lack of standard datasets and evaluation methods makes it hard to compare models fairly, while limited high-quality data affects training and model performance. Many models also struggle with explainability and adaptability, limiting their use in real-world settings. Future work should focus on creating shared datasets to allow better comparisons, developing models that work across different environments, and exploring methods that don't require large labeled datasets. Addressing these issues can lead to more reliable, adaptable, and responsible ML/DL tools for cybersecurity, helping to better protect software systems from vulnerabilities.

# Acknowledgement

Research reported in this publication was supported in part by funding provided by the National Aeronautics and Space Administration (NASA), under award number 80NSSC20M0124, Michigan Space Grant Consortium (MSGC).

# References

F. Alenezi and C. Tsokos. Machine learning approach to predict computer operating systems vulnerabilities. In *2020 International Conference on Computing, Artificial Intelligence and Smart Technology (ICCAIS)*, pages 1–6, 2020. doi: 10.1109/ICCAIS48893.2020.9096731.

Z. Bilgin, M. A. Ersoy, E. U. Soykan, E. Tomur, P. Çomak, and L. Karaçay. Vulnerability prediction from source code using machine learning. *IEEE Access*, 8:150672–150684, 2020.

M. A. Butt, Z. Ajmal, Z. I. Khan, M. Idrees, and Y. Javed. An in-depth survey of bypassing buffer overflow mitigation techniques. *Applied Sciences*, 12(13):6702, 2022.

M. Byun, Y. Lee, and J.-Y. Choi. Analysis of software weakness detection of cbmc based on cwe. In *2020 International Conference on Advanced Communication Technology (ICACT)*, pages 171–175, 2020. doi: 10.23919/ICACT48636.2020.9061281.

S. S. Das, E. Serra, M. Halappanavar, A. Pothen, and E. Al-Shaer. V2w-bert: A framework for effective hierarchical multiclass classification of software vulnerabilities. In *2021 IEEE 8th International Conference on Data Science and Advanced Analytics (DSAA)*, pages 1–12. IEEE, 2021.

K. Filus and J. Domańska. Software vulnerabilities in tensorflow-based deep learning applications. *Computers & Security*, 124:102948, 2023.

H. Hanif and S. Maffeis. Vulberta: Simplified source code pre-training for vulnerability detection. In *2022 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2022.

H. Hanif, M. H. N. M. Nasir, M. F. Ab Razak, A. Firdaus, and N. B. Anuar. The rise of software vulnerability: Taxonomy of software vulnerabilities detection and machine learning approaches. *Journal of Network and Computer Applications*, 179:103009, 2021.

G. Huang, Y. Li, Q. Wang, J. Ren, Y. Cheng, and X. Zhao. Automatic classification method for software vulnerability based on deep neural network. *IEEE Access*, 7:28291–28298, 2019.

T. Ji, Y. Wu, C. Wang, X. Zhang, and Z. Wang. The coming era of alphahacking?: A survey of automatic software vulnerability detection, exploitation and patching techniques. In *2018 IEEE Third International Conference on Data Science in Cyberspace (DSC)*, pages 53–60, 2018. doi: 10.1109/DSC.2018.00017.

S. Kumar and R. R. Mallipeddi. Impact of cybersecurity on operations and supply chain management: Emerging trends and future research directions. *Production and Operations Management*, 31(12):4488–4500, 2022.

G. Li, L. Ren, Y. Fu, Z. Yang, V. Adetola, J. Wen, and Z. O'Neill. A critical review of cyber-physical security for building automation systems. *Annual Reviews in Control*, 55:237–254, 2023.

J. H. Li. Cyber security meets artificial intelligence: A survey. *Frontiers of Information Technology & Electronic Engineering*, 19(12):1462–1474, 2018.

Z. Li, D. Zou, J. Tang, Z. Zhang, M. Sun, and H. Jin. A comparative study of deep learning-based vulnerability detection system. *IEEE Access*, 7:103184–103197, 2019.